Stoquify Internal Activation Gate Remediation and Controlled-Pilot Readiness Execution Report

Execution date: 2026-07-24

Source prompt: `docs/agents-runtime/STOQUIFY_INTERNAL_ACTIVATION_GATE_REMEDIATION_AND_CONTROLLED_PILOT_READINESS_EXECUTION_PROMPT_2026-07-23.md`

Source analysis: `what-next/agents-runtime/STOQUIFY_INTERNAL_ACTIVATION_GATE_UNBLOCKING_ANALYSIS_AND_EXECUTION_PLAN_2026-07-23.md`

Final status: READY FOR REMEDIATION VERIFICATION

Production/internal activation: BLOCKED and not attempted

1. Executive Decision

Stoquify now has an enforceable repository-level release-control plane for the Phase 2A Command Agent. The implementation replaces informal environment-only activation with a tenant-scoped, audit-backed state machine; independent product and security approvals; six acknowledged ownership responsibilities; manifest-bound certification; authenticated reconciliation; Workflow Assurance incidents; retry-safe webhook delivery; mandatory CI certification; and guarded suspension.

The controlled local PostgreSQL migration and persistence smoke passed. The enabled Command Agent also passed authenticated desktop and mobile Playwright certification while its release remained inactive.

Stoquify is not yet ready for a real controlled internal pilot because the following facts remain external and cannot be fabricated:

1. No real product approval or security approval has been recorded for a production-like release.
2. No named real-world rollout, rollback, support, pilot, security-incident, and on-call backup ownership roster has accepted coverage.
3. No deployment-provider evidence proves that the reconciler is scheduled every five minutes with a managed secret or workload identity.
4. No production-like webhook endpoint is configured, and no test alert has been delivered, acknowledged, and escalated to a named owner.
5. The local Playwright certificate is provisional because the implementation is in an uncommitted working tree. The new CI job must rerun from a clean commit and retain its report hash.
6. The full negative and degradation Playwright matrix requested by the prompt is not yet complete.

The repository-addressable foundation is materially complete. The next stage is remediation verification in a production-like non-production environment, not internal activation.

2. Current-State Revalidation

Before remediation, the Command Agent had deterministic, read-only runtime controls, DRAFT/SHADOW definitions, a kill switch, tenant and role allowlists, abandoned-run reconciliation, and a kill-switch Playwright test. It did not have a release package, approval model, ownership model, browser certification record, production alert transport, or a

guarded activation transition. The provisioning script could set definitions to `ACTIVE` using `--activate`.

The current implementation separates these concepts:

Concept	Current result
Implemented	Release schema, service, protected actions, reconciliation, alert worker, browser certification, CI job
Configured locally	E2E pilot tenant, two seeded users, shadow definitions, inactive certification session
Deployed locally	Migration `20260723100000_agent_runtime_release_control` in local PostgreSQL
Tested	TypeScript, static gates, 17 Jest suites, PostgreSQL smoke, desktop/mobile Playwright
Certified locally	E2E package and report hash only; provisional until clean-commit CI rerun
Formally approved	No real product/security approval
Activated	No; `activatedAt` is null and activation is rejected

3. Release-Control Architecture

3.1 State machine

The database now supports:

```
DRAFT -> REVIEWED -> APPROVED -> PROVISIONED_INACTIVE -> PILOT_CERTIFIED ->
ACTIVE_INTERNAL -> SUSPENDED -> RETIRED
```

Implemented guarded transitions cover preparation, review, product/security decisions, owner assignment and acceptance, inactive provisioning, certification, internal activation, and suspension. `RETIRED` exists in the schema; a dedicated retirement command remains to be added before long-term multi-release operations.

3.2 Bound evidence

`AgentActivationPackage` binds:

- Tenant organization and agent key
- Release version and target environment
- Git commit SHA
- Immutable code-owned manifest and SHA-256 manifest hash
- Agent, skill, prompt, tool, input-schema, and output-schema identities
- Provider policy fixed to `none`
- Approved roles
- Activation window
- Residual-risk record
- Optimistic version
- Reconciliation heartbeat and state
- Alert-transport readiness
- Activation and suspension timestamps

Related tables record append-oriented approvals, ownership, and pilot certification evidence.

3.3 Enforcement order

Every Command Agent run now follows this order:

1. Resolve authenticated tenant, actor, roles, permissions, period, and source route.
2. Evaluate kill switch, rollout mode, pilot organization, role, and `dashboard.read`.
3. Authorize the tenant-scoped release package.
4. Validate exact manifest, approval, ownership, window, certification, reconciliation, and alert health for the requested mode.
5. Resolve the matching active definition projection.
6. Execute only the registered read-only digest tool.
7. Persist run, step, evidence, governance, metrics, and audit records.

No agent receives release-control permissions. No direct ledger, filing, payment, stock, payroll, approval, or permission mutation was added.

4. Prisma Schema and Migration

The migration adds:

- `AgentActivationPackage`
- `AgentActivationApproval`
- `AgentActivationOwner`
- `AgentPilotCertification`
- Activation, approval, ownership, and certification enums
- `AGENT_RUNTIME` Workflow Assurance category
- A `PROCESSING` alert-delivery state
- Delivery claim, retry, next-attempt, lock, and external-reference fields

Indexes cover tenant/state lookup, environment/release uniqueness, approval expiry, owner responsibility, certification identity, and due webhook delivery.

Verification:

- Prisma schema validation: passed
- Migration safety gate: `ready`, 8/8 checks, zero destructive findings
- Controlled local deployment: passed
- Migration status: 31 migrations, database up to date
- PostgreSQL persistence and constraint smoke: passed

5. Governed Transitions and RBAC

The canonical service is `services/agents/agent-release-control.service.ts`.

Protected server actions use separate permissions:

- `agent.release.prepare`
- `agent.release.product.approve`
- `agent.release.security.approve`
- `agent.release.owner.manage`
- `agent.release.owner.accept`
- `agent.release.provision`
- `agent.release.activate`
- `agent.release.suspend`

Controls enforced:

- Fresh authentication on preparation, approval, ownership, provisioning, activation, and suspension
- Tenant-scoped package lookup
- Active organization membership for actors and owners
- Different product and security approvers
- Different primary and backup owners
- Owner self-acknowledgement
- Approval expiry and revocation semantics
- Exact manifest matching
- Optimistic package version checks
- Activation-window enforcement
- Complete owner coverage
- Current report-bound certification
- Current reconciliation and alert readiness
- Audit evidence for transitions

6. Direct-Activation Bypass Remediation

`scripts/agent-runtime-phase-2a-provision.js` is now DRAFT/SHADOW only.

It rejects:

- `--activate`
- `--rollout=internal`

`scripts/agent-release-control-gate.js` fails if direct activation returns, runtime authorization is reordered, required controls disappear, enabled-pilot CI is removed, or certification code gains activation authority.

The existing prohibited-action gate was extended only for named release metadata and Workflow Assurance delegates. Business-domain mutation remains prohibited.

7. Approval and Ownership Workflow

Product and security approvals are separate records with approver identity, decision, evidence hash, risk acceptance, decision time, expiry, and revocation time.

Each of these owner responsibilities is mandatory:

Responsibility	Required evidence
Rollout	Primary, backup, accepted coverage, escalation route, runbook version
Rollback	Primary, backup, accepted coverage, escalation route, runbook version
Support	Primary, backup, accepted coverage, escalation route, runbook version
Pilot	Primary, backup, accepted coverage, escalation route, runbook version
Security incident	Primary, backup, accepted coverage, escalation route, runbook version
On-call backup	Primary, backup, accepted coverage, escalation route, runbook version

The E2E records prove the mechanism only. They are explicitly non-production and do not satisfy the human gate.

8. Reconciliation and Heartbeat

The authenticated internal endpoint now performs three bounded operations:

1. Reconcile abandoned agent runs.
2. Reconcile release packages, definitions, evidence, and runtime health.
3. Dispatch due Workflow Assurance webhook deliveries.

The control-plane reconciler checks manifest identity, approvals, owners, certification, definition status/mode, active-release uniqueness, activation window, heartbeat, and alert transport. It uses Stoquify's canonical digest-bound Workflow Assurance persistence, producing check runs, findings, incidents, events, in-app alerts, webhook deliveries, and audit evidence.

Authentication uses a timing-safe bearer secret with a 32-character minimum. The provider-neutral schedule target is:

- Method: `POST`
- Route: `/api/internal/agents/reconcile-abandoned`
- Frequency: every five minutes
- Authentication: `Authorization: Bearer <managed secret>`
- Rotation: overlap old/new scheduler deployments, verify a successful heartbeat, then revoke the old value

No deployment provider was discoverable, so schedule deployment is correctly reported as pending.

9. Alert Delivery

`services/assurance/assurance-alert-delivery.service.ts` implements:

- Atomic `PENDING -> PROCESSING` claim
- Stale-lock recovery

- Bounded batches
- Five attempts with exponential backoff
- Dedupe and idempotency keys
- Timeout control
- Safe redacted payloads
- Delivered/failed persistence
- External request reference capture
- Production HTTPS and secret-strength validation

The local smoke intentionally ran without `STOQUIFY_ASSURANCE_ALERT_WEBHOOK_URL`. It correctly persisted an incident and pending webhook, marked alert transport unhealthy, and blocked activation.

Before pilot approval, a production-like transport must demonstrate delivery, acknowledgement, retry, failure exhaustion, and escalation to a named owner.

10. Enabled-Pilot Certification

The certification runner:

1. Provisions only DRAFT/SHADOW definitions.
2. Creates a tenant-scoped E2E package.
3. Records two distinct E2E approvals.
4. Assigns and acknowledges all six owner responsibilities.
5. Moves the package to `PROVISIONED_INACTIVE` through the canonical service.
6. Opens a certification-only session restricted to environment `e2e`, outside production.
7. Runs real authenticated desktop and mobile Playwright projects against PostgreSQL and protected actions.
8. Hashes the JSON report.
9. Records commit SHA, manifest hash, suite version, CI run ID, report hash, pass time, and expiry.
10. Verifies that no activation timestamp exists.

Results:

- Authentication setup: 2 passed
- Enabled desktop: passed
- Enabled mobile: passed
- Total: 4 passed, 0 unexpected, 0 flaky
- Duration: 128.157 seconds
- Package state: `PILOT_CERTIFIED`
- `activatedAt`: null

The CI workflow now contains `command-agent-enabled-pilot`, installs Chromium, uses PostgreSQL, executes the governed certification runner, and uploads JSON, state, trace, screenshot, video, and HTML evidence when available.

11. PostgreSQL Evidence

Bounded smoke artifact:
`what-next/agents-runtime/command-agent-release-control-postgres-smoke.json`

Observed result:

- Approval records: 2
- Distinct approvers: 2
- Accepted owner responsibilities: 6
- Browser certifications: 1
- Release audit records: 18
- Reconciliation packages scanned: 1
- Workflow Assurance incidents: 1
- Alert readiness: false, `webhook_url_missing`
- Delivery readiness: false, `webhook_url_missing`
- Activation attempt: rejected with `RELEASE_RECONCILIATION_STALE`
- Release state after attempted activation: `PILOT_CERTIFIED`
- Activated timestamp: null

12. Verification Register

Verification	Result
Prisma validate	Passed
Migration safety	Passed, 8/8
Local migration deploy	Passed
Migration status	Up to date
Full TypeScript check	Passed
Focused ESLint	Passed; one pre-existing anonymous-default-export warning in permission config
Agent tool-registry gate	Passed
Agent prohibited-action gate	Passed
Agent release-control gate	Passed
Phase 2A Command Agent gate	Passed
Agent and component Jest	17 suites, 48 tests passed
Enabled Playwright	4 tests passed
PostgreSQL release smoke	Passed
Production alert delivery	Not run; endpoint/secret absent
Remote CI certification	Not run in this local workspace
Production scheduler deployment	No evidence available

13. Controlled Activation Runbook

1. Merge the implementation and obtain a clean commit SHA.
2. Deploy the migration and application to an isolated production-like environment.
3. Configure the release environment, kill switch, rollout allowlists, reconciler identity, and alert transport from managed secrets.
4. Create a release package bound to the deployed commit and exact manifest.
5. Record real product approval and real security approval from different authorized users.
6. Assign all six responsibilities with different primary/backup users; each primary accepts current coverage.
7. Provision inactive definitions through the canonical transition.
8. Run the full CI certification matrix and persist its report hash.
9. Start the five-minute reconciler and confirm fresh passing heartbeats.
10. Send a production-like alert; confirm delivery, acknowledgement, escalation, and external reference.
11. Confirm zero open blocking Agent Runtime incidents.
12. Review residual risks and activation window.
13. Obtain a separately recorded go decision.
14. Activate one internal pilot organization through `agent.release.activate` with fresh authentication.
15. Monitor run failure, denial, stale-output, heartbeat, and alert-delivery objectives.

14. Rollback and Emergency Disable

1. Set `STOQUIFY_COMMAND_AGENT_KILL_SWITCH=1` for immediate execution denial.
2. Invoke the guarded suspension transition for the active package.
3. Confirm definitions project to `PAUSED/SHADOW` and the package becomes `SUSPENDED`.
4. Run reconciliation and retain the resulting assurance evidence.
5. Notify rollout, rollback, support, pilot, and security owners.
6. Preserve runs, evidence, incidents, deliveries, and audit logs; do not delete evidence.
7. Diagnose manifest, approval, scope, runtime, or transport failure.
8. Require a new package/certification if code, manifest, prompt, schema, permission, or deployment identity changes.

15. Human and Deployment Completion Register

Required completion	Required authority/evidence	Current state
Product approval	Authorized product approver, evidence hash, expiry	Missing
Security approval	Different authorized security approver, evidence hash, expiry	Missing
Six owner assignments	Directory identities, backups, accepted coverage, runbooks	Missing for real pilot
Scheduler deployment	Platform owner, five-minute job, managed identity/secret, successful heartbeat	Missing
Secret rotation	Security/platform owner, managed secret reference and rotation test	Missing
Alert transport	SRE/security owner, HTTPS endpoint, secret, delivery evidence	Missing
Alert acknowledgement	Named on-call owner and escalation evidence	Missing

Required completion	Required authority/evidence	Current state
Clean-commit CI certificate	GitHub Actions run and retained report hash	Missing
Branch protection	Required status check for enabled-pilot job	Missing evidence
Full negative matrix	Security/QA evidence for tenant, role, permission, draft, mismatch, suspension, rollback	Incomplete

16. Final Gate Matrix

Gate	Technical implementation	Local evidence	External completion	Final status
Product/security approval	Complete	Two E2E identities	Real decisions absent	BLOCKED
Named ownership	Complete	Six E2E responsibilities	Real roster absent	BLOCKED
Reconciliation	Complete	PostgreSQL check/incident passed	Schedule/managed secret not deployed	BLOCKED
Alerting	Worker complete	Fail-closed pending delivery proved	No delivered/acknowledged alert	BLOCKED
Enabled pilot	Desktop/mobile and CI job complete	4/4 local pass	Clean-commit CI and full matrix pending	BLOCKED
Direct activation prevention	Complete	Static gates passed	None	PASSED
Read-only business boundary	Complete	Prohibited-action gate passed	None	PASSED
Controlled database migration	Complete	Local deploy/status passed	Production-like deploy pending	READY FOR VERIFICATION

17. Residual Technical Work

The following repository-addressable work should be completed before declaring the broader prompt fully implemented:

1. Add a guarded `RETIRED` transition and retention policy.
2. Add explicit approval/certification command idempotency keys and replay receipts.
3. Add deterministic retry jitter and a dead-letter operational view for exhausted webhook deliveries.
4. Add automatic guarded suspension for active releases on blocking reconciliation drift.
5. Complete browser-negative coverage for unauthorized tenant, unauthorized role, missing permission, module denial, DRAFT definition, manifest mismatch, suspension, rollback, keyboard, and accessibility.
6. Add degradation browser cases for stale, partial, empty, timeout, retry, and replay behavior.
7. Add database assertions around prohibited business-table mutation and cross-tenant non-exposure.
8. Resolve the global definition-projection limitation before simultaneous multi-tenant releases with different lifecycle states.
9. Add a dedicated Agent Runtime operations dashboard or explicit filtered view in the existing Assurance Control Tower.

18. Final Decision

Gates passed: direct-activation prevention, read-only business boundary, release-control schema, controlled local migration, tenant-safe transition service, local PostgreSQL persistence, local enabled desktop/mobile certification.

Gates blocked: real approvals, real owner roster, deployed reconciliation schedule, managed scheduler authentication, delivered and acknowledged production-like alert, clean-commit CI certificate, full negative/degradation matrix.

Ready for controlled internal pilot: No.

May advance beyond Phase 2A: It may advance to production-like remediation verification, but not to internal activation or general pilot rollout.

Next technically executable and authorized action: Commit and review the implementation, deploy it to an isolated production-like environment with managed scheduler and webhook credentials, record real approvers/owners, run the full required CI matrix, and rerun reconciliation until the package is `PILOT_CERTIFIED` with `PASSED` reconciliation, healthy alert transport, no blocking incidents, and a separately authorized activation decision.